

Roteiro Pratico - Aula 2

Evolucao do cadastro de alunos em Java Desktop

Antes de comecar

Nesta pratica, voce nao vai apenas repetir a criacao de uma classe. A proposta e evoluir o projeto da primeira pratica para um pequeno cadastro desktop em memoria, com entrada de dados, listagem, busca e validacoes simples.

Produto esperado

Ao final, o projeto deve abrir um menu com JOptionPane, cadastrar varios alunos, listar os alunos cadastrados e buscar um aluno pelo nome.

Passo 1 - Abrir ou criar o projeto

- Abra o Eclipse e use o workspace da disciplina.
- Abra o projeto usado na primeira pratica ou crie um Java Project chamado Aula02CadastroAlunos.
- Crie ou confira o pacote br.ceub.desenvolvimento.aula02 dentro de src.
- Se estiver reaproveitando o projeto anterior, mantenha a classe Aluno e ajuste o codigo conforme o passo 2.

Passo 2 - Ajustar a classe Aluno

A classe Aluno continua sendo o modelo de dominio. Ela guarda os dados de um aluno e devolve um resumo. Nesta versao, vamos adicionar getters para permitir buscas sem deixar os atributos publicos.

```
package br.ceub.desenvolvimento.aula02;

public class Aluno {
    private String nome;
    private int semestre;
    private String curso;

    public Aluno(String nome, int semestre, String curso) {
        this.nome = nome;
        this.semestre = semestre;
        this.curso = curso;
    }

    public String getNome() {
        return nome;
    }

    public int getSemestre() {
        return semestre;
    }

    public String getCurso() {
        return curso;
    }

    public String gerarResumo() {
        return nome + " - " + curso + " - " + semestre + " semestre";
    }
}
```

Passo 3 - Criar a classe CadastroAlunos

A classe CadastroAlunos sera responsavel por guardar varios objetos Aluno. Ela usa ArrayList para manter os dados enquanto o programa esta aberto.

```
package br.ceub.desenvolvimento.aula02;

import java.util.ArrayList;

public class CadastroAlunos {
    private ArrayList<Aluno> alunos = new ArrayList<>();

    public void adicionar(Aluno aluno) {
        alunos.add(aluno);
    }

    public String listarTodos() {
        if (alunos.isEmpty()) {
            return "Nenhum aluno cadastrado.";
        }

        String resultado = "";
        for (Aluno aluno : alunos) {
            resultado += aluno.gerarResumo() + "\n";
        }
        return resultado;
    }

    public Aluno buscarPorNome(String nome) {
        for (Aluno aluno : alunos) {
            if (aluno.getNome().equalsIgnoreCase(nome)) {
                return aluno;
            }
        }
        return null;
    }

    public int quantidade() {
        return alunos.size();
    }
}
```

Passo 4 - Criar o menu desktop com JOptionPane

A classe Principal controla a interacao com o usuario. Ela mostra o menu, le os dados, cria objetos Aluno e chama os metodos da classe CadastroAlunos.

```
package br.ceub.desenvolvimento.aula02;

import javax.swing.JOptionPane;

public class Principal {
    public static void main(String[] args) {
        CadastroAlunos cadastro = new CadastroAlunos();
        int opcao = -1;

        while (opcao != 0) {
            String menu = "1 - Cadastrar aluno\n"
                + "2 - Listar alunos\n"
                + "3 - Buscar aluno por nome\n"
                + "0 - Sair";

            opcao = lerInteiro(menu, 0, 3);

            if (opcao == 1) {
                String nome = lerTextoObrigatorio("Nome do aluno:");
                String curso = lerTextoObrigatorio("Curso:");
                int semestre = lerInteiro("Semestre:", 1, 10);

                Aluno aluno = new Aluno(nome, semestre, curso);
            }
        }
    }
}
```

```

        cadastro.adicionar(aluno);
        mostrar("Aluno cadastrado com sucesso.");
    } else if (opcao == 2) {
        mostrar(cadastro.listarTodos());
    } else if (opcao == 3) {
        String nomeBusca = lerTextoObrigatorio("Nome para busca:");
        Aluno encontrado = cadastro.buscarPorNome(nomeBusca);

        if (encontrado == null) {
            mostrar("Aluno nao encontrado.");
        } else {
            mostrar(encontrado.gerarResumo());
        }
    }
}

mostrar("Programa encerrado. Total de alunos: " + cadastro.quantidade());
}

private static String lerTextoObrigatorio(String mensagem) {
    String texto = "";
    while (texto == null || texto.trim().isEmpty()) {
        texto = JOptionPane.showInputDialog(mensagem);
        if (texto == null || texto.trim().isEmpty()) {
            mostrar("Campo obrigatorio.");
        }
    }
    return texto.trim();
}

private static int lerInteiro(String mensagem, int minimo, int maximo) {
    while (true) {
        try {
            String entrada = JOptionPane.showInputDialog(mensagem);
            int numero = Integer.parseInt(entrada);

            if (numero >= minimo && numero <= maximo) {
                return numero;
            }
            mostrar("Digite um numero entre " + minimo + " e " + maximo + ".");
        } catch (NumberFormatException erro) {
            mostrar("Digite apenas numeros.");
        }
    }
}

private static void mostrar(String mensagem) {
    JOptionPane.showMessageDialog(null, mensagem);
}
}

```

Passo 5 - Executar e testar

- Clique com o botao direito em Principal.java.
- Escolha Run As > Java Application.
- Cadastre pelo menos tres alunos.
- Liste todos os alunos cadastrados.
- Busque um aluno existente e um aluno inexistente.
- Teste entradas invalidas: nome em branco, semestre com texto e semestre fora da faixa.

Passo 6 - Registrar o que mudou

Antes	Agora
Dois objetos fixos no codigo	Varios objetos criados durante a execucao
Saida no Console	Interacao por caixas de dialogo
Sem lista	ArrayList guardando os alunos em memoria

Sem busca	Busca por nome percorrendo a lista
Sem validacao	Campos obrigatorios e semestre dentro de uma faixa

Checklist de conclusao

Verificacao	OK
Classe Aluno com atributos privados, construtor, getters e gerarResumo	
Classe CadastroAlunos criada com ArrayList	
Metodo adicionar funcionando	
Metodo listarTodos tratando lista vazia	
Metodo buscarPorNome retornando aluno ou null	
Classe Principal com menu JOptionPane	
Validacao de texto obrigatorio e numero	
Projeto executando sem erro no Eclipse	

Erros frequentes

Problema	Possivel causa	Como resolver
ArrayList em vermelho	Import ausente	Adicionar import java.util.ArrayList;
JOptionPane em vermelho	Import ausente	Adicionar import javax.swing.JOptionPane;
NullPointerException ao cancelar janela	Entrada cancelada sem tratamento	Nesta pratica, evite cancelar as janelas durante os testes.
NumberFormatException	Texto digitado onde deveria haver numero	Usar o metodo lerInteiro com try/catch.
Busca nao encontra nomes iguais	Comparacao com ==	Usar equalsIgnoreCase para comparar textos.

Desafio adicional

Acrescente uma quarta opcao no menu: buscar alunos por semestre. O usuario deve digitar um semestre e o sistema deve mostrar todos os alunos daquele semestre ou uma mensagem informando que nenhum aluno foi encontrado.